



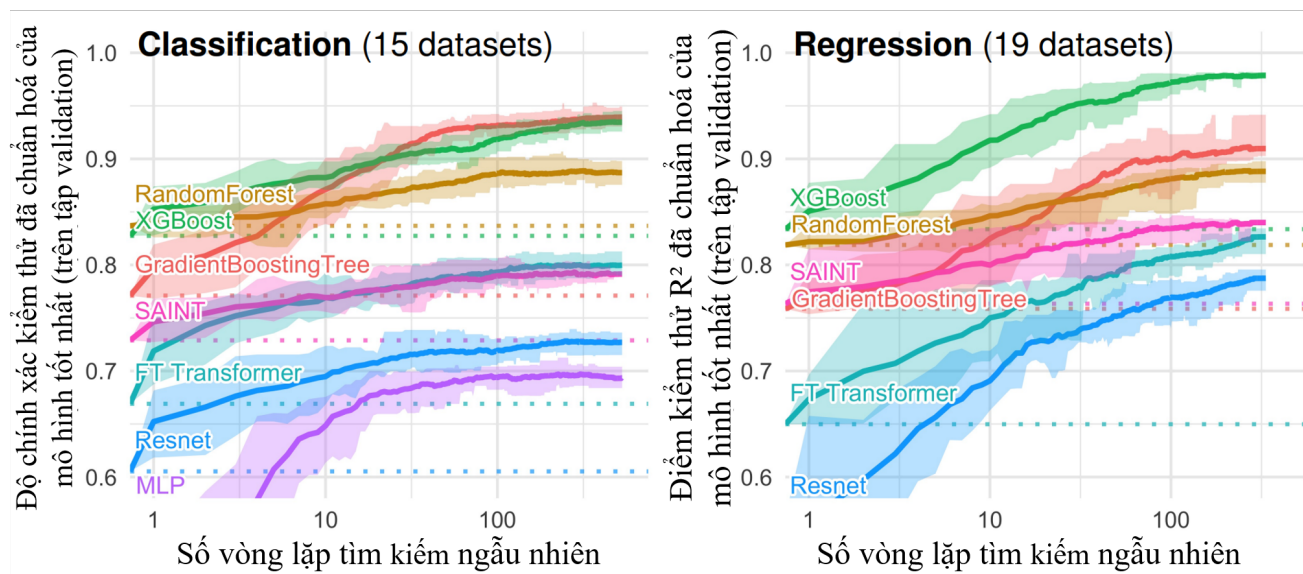
AI VIET NAM – AI COURSE 2026

Step-by-Step: XGBoost

Nguyễn Anh Khôi Dương Đình Thắng Đinh Quang Vinh

I. Dẫn nhập

Deep Learning (tạm dịch: học sâu) có thể thống trị trong dữ liệu ảnh và ngôn ngữ, nhưng khi bước vào thế giới dữ liệu bảng, một cái tên không thể không nhắc đến đó là **XGBoost**. Trên 45 bộ dữ liệu cỡ trung, khoảng 10.000 mẫu mỗi bộ, với hơn 20.000 giờ tối ưu các siêu tham số, công trình nghiên cứu của Léo Grinsztajn và cộng sự [1] đã chỉ ra rằng XGBoost không chỉ vượt trội về hiệu năng mà còn giữ vững độ ổn định ngay cả khi dữ liệu chứa nhiều đặc trưng dư thừa, giá trị phức tạp khó dự đoán, hoặc cấu trúc không thuận lợi cho mạng sâu. Chính sự bền bỉ này khiến XGBoost trở thành chuẩn mực vàng cho học máy trên dữ liệu bảng.



Hình 1: XGBoost đạt kết quả ấn tượng trên 15 bộ dữ liệu phân loại và 19 bộ dữ liệu hồi quy, vượt trội so với cả các mô hình cây khác lẫn các mạng sâu như MLP, ResNet, Transformer. Kết quả này cho thấy, ngay cả khi các mô hình học sâu đang tiến bộ, các mô hình dạng cây Boosting vẫn duy trì hiệu năng và độ ổn định vượt bậc (nguồn [1]).

Mục lục

I.	Dẫn nhập	1
II.	Thuật toán XGBoost	4
II.1.	Tổng quan về Boosting	4
II.2.	XGBoost	5
II.3.	Mục tiêu tối ưu	8
II.4.	Mở rộng Taylor bậc hai	9
II.5.	Trọng số nút lá và Gain	10
II.6.	Các bước tính toán tổng quát	11
II.7.	Bài toán cụ thể	12
III.	Thực hành	16
III.1.	Mô tả dữ liệu Heart Disease	16
IV.	Câu hỏi trắc nghiệm	20
V.	Tài liệu tham khảo	22
	Phụ lục	23

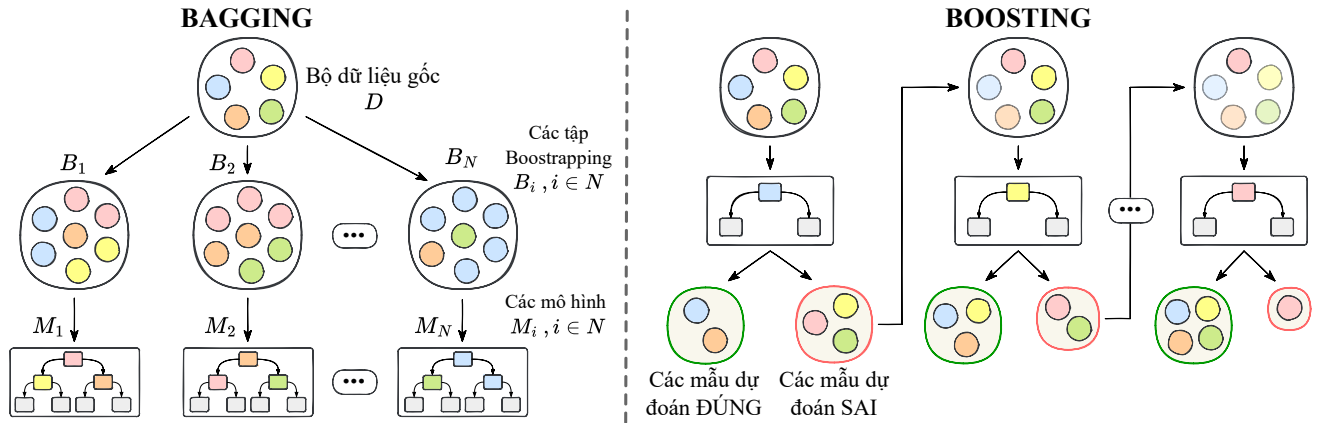
Bảng tra cứu thuật ngữ

Thuật ngữ	Giải thích
Objective Function	Hàm mục tiêu, hàm cần tối ưu.
Loss Function	Hàm mất mát, thước đo sai số giữa dự đoán và giá trị thực.
Gradient Descent	Thuật toán tối ưu lặp cập nhật tham số theo hướng âm của gradient để giảm hàm mất mát.
Regularization	Điều chuẩn, kỹ thuật thêm ràng buộc để tránh mô hình học quá khớp dữ liệu huấn luyện.
Gradient	Đạo hàm bậc nhất, cho biết hướng thay đổi nhanh nhất của hàm.
Hessian	Đạo hàm bậc hai, cho biết độ cong của hàm.
Gain	Điểm lợi ích, mức giảm lỗi khi tách một nhánh trong cây quyết định.
Pruning	Cắt tỉa cây, loại bỏ các nhánh không cần thiết để tránh quá khớp.
Overfitting	Quá khớp, khi mô hình học quá chi tiết vào dữ liệu huấn luyện và kém tổng quát.
Subsample	Lấy mẫu ngẫu nhiên theo hàng, chọn một phần dữ liệu huấn luyện để giảm overfitting.
Colsample	Lấy mẫu ngẫu nhiên theo cột, chọn ngẫu nhiên các đặc trưng để huấn luyện.
Sparsity-Aware	Xử lý dữ liệu thưa, kỹ thuật tối ưu khi dữ liệu có nhiều giá trị rỗng/0.
Pseudo-Residual	Phần dư giả, sai số tạm thời được tính ở từng vòng lặp trong boosting.
Hyperparameter	Siêu tham số, tham số do người dùng chọn trước khi huấn luyện.
Learning Rate	Hệ số học, tốc độ cập nhật trọng số trong quá trình tối ưu.

II. Thuật toán XGBoost

II.1. Tổng quan về Boosting

Boosting (tạm dịch: tăng cường) là một kỹ thuật học máy thuộc nhóm ensemble (tạm dịch: tổ hợp mô hình), kết hợp nhiều weak learners (tạm dịch: mô hình yếu), thường là shallow decision trees (tạm dịch: cây quyết định nông) theo cách tuần tự phụ thuộc lẫn nhau để tạo nên một mô hình tổng thể mạnh hơn, qua đó giảm sai số dự đoán và cải thiện khả năng tổng quát hoá.



Hình 2: Minh họa hai phương pháp Ensemble: Bagging và Boosting.

Khác với Bagging (tạm dịch: lấy mẫu và gộp), các mô hình này được huấn luyện một cách song song rồi gộp bằng cách lấy giá trị trung bình hoặc biểu quyết từ số đông, trong khi đó, Boosting huấn luyện từng mô hình một cách tuần tự, nghĩa là mỗi mô hình đến sau tập trung khắc phục các sai sót còn lại của toàn bộ các mô hình trước đó, được minh họa như Hình 2). Kết quả cuối cùng của mô hình Boosting thường là weighted aggregation (tạm dịch: gộp có trọng số) của tất cả mô hình yếu, không chỉ riêng dự đoán của mô hình cuối cùng:

$$F_M(x) = \sum_{m=1}^M \alpha_m h_m(x).$$

Trong đó:

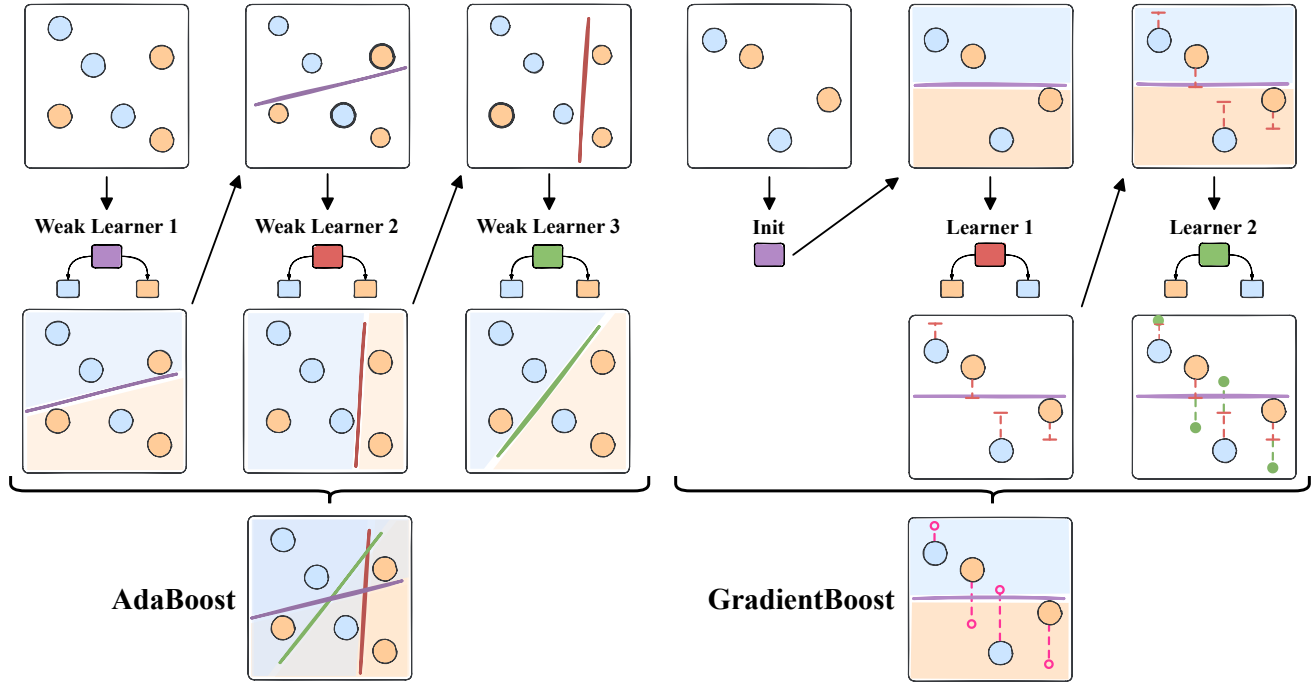
- h_m là mô hình yếu ở vòng m ;
- α_m là trọng số.

Với bài toán classification (tạm dịch: phân loại), cách lấy tổ hợp thường là weighted voting (tạm dịch: biểu quyết có trọng số); còn đối với bài toán regression (tạm dịch: hồi quy), tổ hợp nói trên sẽ là weighted sum (tạm dịch: tổng có trọng số).

Về họ thuật toán tăng cường, **AdaBoost** [2] (Adaptive Boosting) là mô hình khởi đầu tiêu biểu với thuật toán gán sample weights (tạm dịch: trọng số cho mẫu) lớn hơn cho các mẫu bị dự đoán sai để mô hình kế tiếp tập trung nhiều hơn vào những mẫu khó này. Qua nhiều vòng lặp, các

mô hình yếu được gộp lại với trọng số riêng phản ánh chất lượng của chúng. Thuật toán được minh họa ở Hình 3 phía bên trái.

Tiếp đó, **Gradient Boosting** (tạm dịch: tăng cường theo gradient) tổng quát hoá cơ chế học bằng cách xem quá trình tăng cường là bài toán tối ưu hoá một hàm mất mát bất kỳ thông qua phương pháp gradient descent. Mỗi mô hình yếu được huấn luyện để khớp các pseudo-residuals (tạm dịch: phần dư giả). Cách nhìn “tối ưu hoá theo hàm mất mát” này mang lại tính linh hoạt cao, áp dụng được cho nhiều bài toán như classification, regression và là nền tảng cho các biến thể hiện đại hiệu quả hơn.



Hình 3: Minh họa 2 thuật toán Boosting: AdaBoost và GradientBoost, trong bài toán phân loại.

II.2. XGBoost

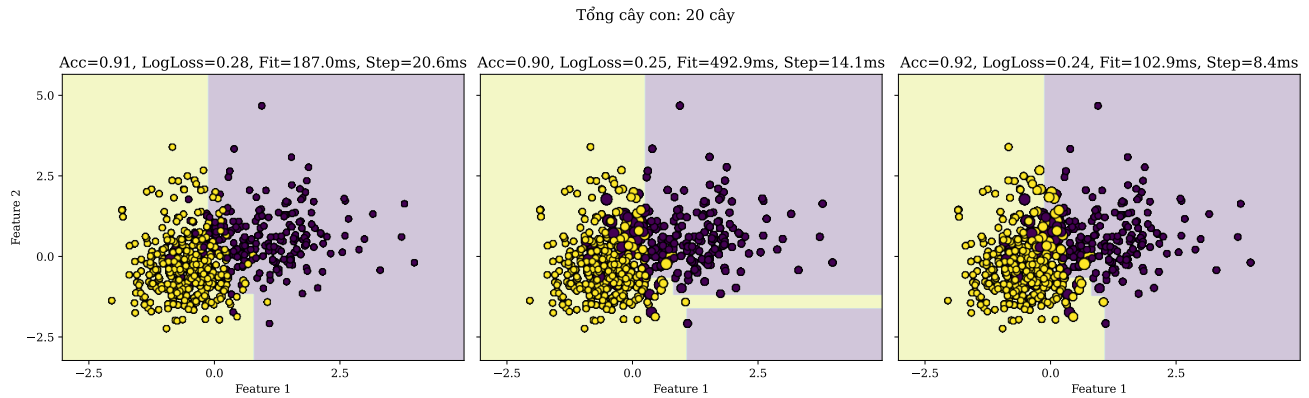
Tiếp nối Gradient Boosting (GB) đã trình bày, **XGBoost** (eXtreme Gradient Boosting) kế thừa ý tưởng tối ưu hoá theo hàm mất mát và được phát triển để giải quyết hai hạn chế của thuật toán Gradient Boosting truyền thống:

- Tăng tốc độ huấn luyện và cải thiện scalability (tạm dịch: khả năng mở rộng).
- Nâng độ chính xác nhờ regularization (tạm dịch: điều chuẩn) và tree pruning (tạm dịch: cắt tỉa cây) hợp lý.

Thuật toán được giới thiệu năm 2016 bởi Tianqi Chen và Carlos Guestrin [3], nhanh chóng phổ biến trong các cuộc thi khoa học dữ liệu. Ở mức hệ thống và thuật toán, XGBoost bổ sung nhiều kỹ thuật giúp mô hình vừa nhanh vừa ổn định:

- Regularization L1/L2 (tạm dịch: điều chuẩn L1/L2) trực tiếp vào hàm mục tiêu để kiểm soát độ phức tạp.

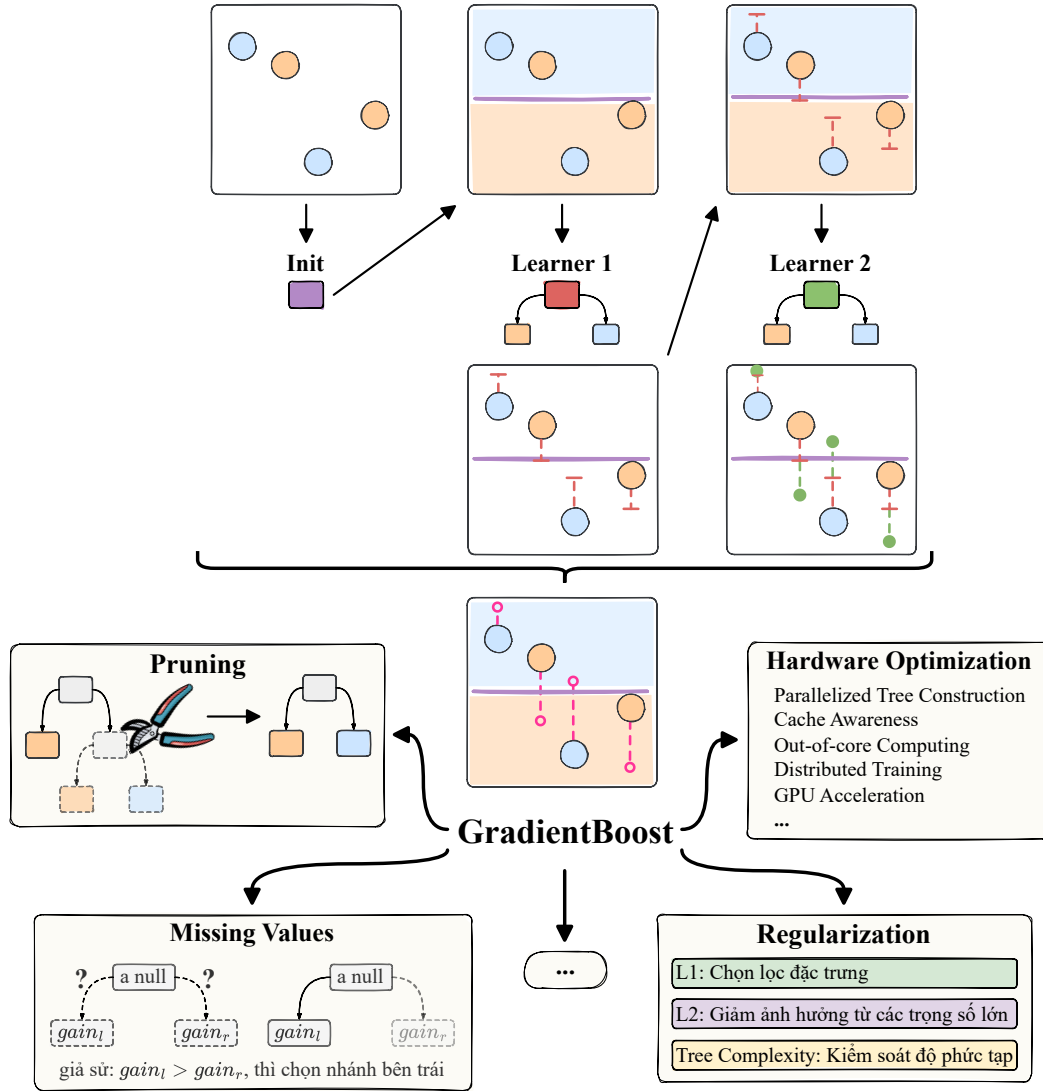
- Shrinkage/Learning rate (tạm dịch: hệ số học) để cập nhật thận trọng theo từng vòng cộng dồn.
- Column subsampling (tạm dịch: lấy mẫu theo cột) nhằm giảm tương quan và tăng tốc.
- Sparsity-aware (tạm dịch: nhận biết dữ liệu thưa), tự động chọn nhánh mặc định cho các giá trị bị thiếu.
- Parallelization, out-of-core, distributed training (tạm dịch: song song hoá, huấn luyện ngoài bộ nhớ, huấn luyện phân tán) để xử lý trên dữ liệu lớn.



Hình 4: So sánh AdaBoost, Gradient Boosting và XGBoost trên bài toán phân loại nhị phân với 2 đặc trưng và khoảng 17000 mẫu. XGBoost đạt kết quả tốt nhất về Acc (Accuracy - tạm dịch: độ chính xác), LogLoss (tạm dịch: giá trị mất mát), tổng thời gian huấn luyện (Fit) và thời gian mỗi bước (Step). Kích thước điểm biểu diễn trọng số hoặc mức lỗi của từng mẫu. [Xem mã nguồn](#).

Về cơ chế học, XGBoost vẫn bám sát tinh thần cộng dồn theo từng vòng của Gradient Boosting, mỗi bước huấn luyện một cây nhỏ để khớp các pseudo-residuals, rồi cộng dồn dự đoán để giảm dần sai số và hạn chế overfitting (tạm dịch: quá khớp).

Ở mỗi vòng lặp, mô hình chỉ cập nhật một bước nhỏ hệ số học theo hướng giảm lỗi hiệu quả nhất, giúp tiến chậm mà chắc. Đồng thời, các cơ chế điều chuẩn và cắt tỉa cây giữ cho mô hình gọn, ổn định và tránh học quá mức ngay cả khi số vòng lặp tăng.



Hình 5: Hình minh hoạ thuật toán XGBoost cho bài toán phân loại.

Có ba cải tiến cốt lõi ở mức công thức thuật toán giúp XGBoost vượt trội hơn so với các mô hình AdaBoost và Gradient Boosting:

- Khai triển Taylor bậc hai của hàm mất mát, tận dụng cả gradient và hessian thay vì chỉ gradient.
- Thêm regularization vào objective (tạm dịch: hàm mục tiêu) để phạt số lượng lá và độ lớn trọng số ở các lá, giúp kiểm soát độ phức tạp và tăng khả năng tổng quát.
- Tối ưu quyết định phân nhánh bằng công thức gain (tạm dịch: điểm lợi ích), kèm tham số γ để cắt tỉa các nhánh không mang lại cải thiện.

Nhờ các thành phần trên, XGBoost thường huấn luyện nhanh hơn, mở rộng tốt hơn và tổng quát hoá ổn định hơn so với GB thuần tuý.

Trước khi mô tả chi tiết, chúng ta thống nhất các ký hiệu toán học như bảng dưới đây.

Bảng tra cứu ký hiệu toán học

Ký hiệu	Ý nghĩa
N	Tổng số mẫu trong tập dữ liệu huấn luyện.
M	Tổng số cây (vòng lặp) của mô hình.
$\mathcal{L}$	Hàm mục tiêu, gồm hàm mất mát l và thành phần điều chuẩn Ω .
$l(y_i, \hat{y}_i)$	Hàm mất mát (ví dụ: MSE) đo sai khác giữa dự đoán và nhãn thật.
f_t	Cây quyết định (decision tree) được thêm ở vòng lặp t .
$\hat{y}_i^{(t)}$	Dự đoán cho điểm dữ liệu i tại vòng lặp t .
$\Omega(f)$	Hàm điều chuẩn: $\Omega(f) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2$, phạt số lá và trọng số lớn.
T	Số lá (leaf nodes) của cây quyết định.
w_j	Giá trị dự đoán tại lá j .
γ	Siêu tham số điều chuẩn, phạt số lượng lá (complexity cost).
λ	Siêu tham số điều chuẩn, phạt độ lớn trọng số (L2 regularization).
g_i	Gradient (đạo hàm bậc nhất của hàm mất mát theo dự đoán).
h_i	Hessian (đạo hàm bậc hai của hàm mất mát theo dự đoán).
G_j	Tổng gradient trong một nút lá: $G_j = \sum_{i \in I_j} g_i$.
H_j	Tổng hessian trong một nút lá: $H_j = \sum_{i \in I_j} h_i$.
I_j	Tập các chỉ số của điểm dữ liệu rơi vào lá j .
w_j^*	Trọng số tối ưu của lá j , $w_j^* = -\frac{\sum_{i \in I_j} g_i}{\sum_{i \in I_j} h_i + \lambda}$.
$\mathcal{L}_{\text{leaf}}$	Hàm mất mát xấp xỉ sau khi thế w_j^* , biểu diễn mức giảm lỗi của cây.
η	Hệ số học (learning rate, shrinkage), nhân thêm vào giá trị của cây mới để làm chậm tốc độ học.

II.3. Mục tiêu tối ưu

Giống như GB, XGBoost cũng học mô hình theo từng vòng lặp bằng cách bổ sung một cây mới f_t để sửa sai so với dự đoán trước đó. Điểm khác biệt là **hàm mục tiêu** trong XGBoost không chỉ có **hàm mất mát** mà còn kèm thêm **điều chuẩn** để kiểm soát độ phức tạp của cây.

$$\mathcal{L}^{(t)} = \sum_{i=1}^n l(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)) + \Omega(f_t), \quad (1)$$

trong đó l là hàm mất mát (ví dụ: sai số bình phương, logistic), $\hat{y}_i^{(t-1)}$ là dự đoán trước vòng t , và $\Omega(f_t)$ là thành phần điều chuẩn:

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2, \quad (2)$$

với T là số lá của cây f , w_j là giá trị tại lá j , còn γ và λ là các siêu tham số. Điều này kế thừa trực tiếp từ tư tưởng của GB (tối ưu hàm mất mát theo bước), nhưng mở rộng thêm thành phần Ω để tránh quá khớp và chọn cấu trúc cây gọn gàng hơn.

II.4. Mở rộng Taylor bậc hai

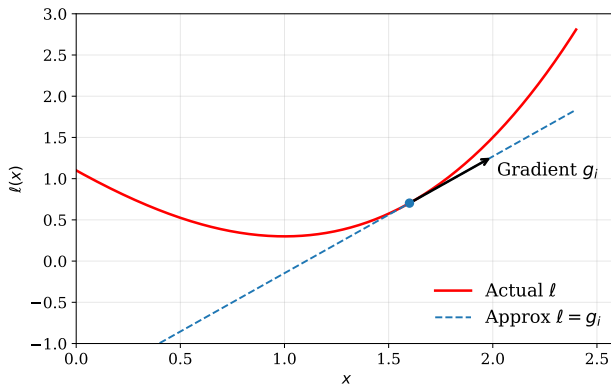
Trong GB truyền thống, ta dùng **đạo hàm bậc nhất** của hàm mất mát để định hướng cách cập nhật. XGBoost tiến thêm một bước là dùng **khai triển Taylor bậc hai** quanh dự đoán cũ $\hat{y}_i^{(t-1)}$. Việc sử dụng khai triển Taylor giúp xấp xỉ một hàm mất mát phức tạp bất kỳ bằng một hàm bậc hai đơn giản hơn tại mỗi bước. Việc tối ưu một hàm bậc hai thì dễ dàng và hiệu quả hơn rất nhiều về mặt tính toán. Nhờ vậy, XGBoost tận dụng cả **độ dốc** (gradient - đạo hàm bậc nhất) và **độ cong** (hessian - đạo hàm bậc hai), giúp việc chọn cách phân nhánh và tính trọng số lá chính xác hơn, thay vì chỉ dựa vào hướng giảm như GB. Hàm mục tiêu sau khi triển khai Taylor bậc hai như sau:

$$\mathcal{L}^{(t)} \approx \sum_{i=1}^n \left[l(y_i, \hat{y}_i^{(t-1)}) + g_i f_t(x_i) + \frac{1}{2} h_i f_t(x_i)^2 \right] + \Omega(f_t), \quad (3)$$

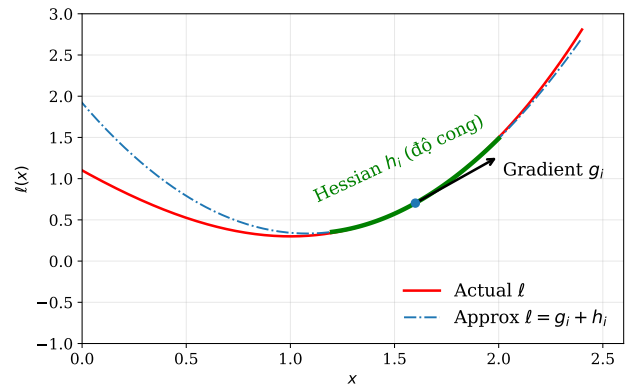
với

$$g_i = \left. \frac{\partial l(y_i, \hat{y})}{\partial \hat{y}} \right|_{\hat{y}=\hat{y}_i^{(t-1)}}, \quad h_i = \left. \frac{\partial^2 l(y_i, \hat{y})}{\partial \hat{y}^2} \right|_{\hat{y}=\hat{y}_i^{(t-1)}}.$$

Trong đó g_i và h_i là gradient và hessian của hàm mất mát l .



Hình 6: Trục quan hàm mất mát (màu đỏ) và hàm xấp xỉ hàm mất mát theo công thức Taylor bậc 1 (màu xanh biển).



Hình 7: Trục quan hàm mất mát (màu đỏ) và hàm xấp xỉ hàm mất mát theo công thức Taylor bậc 2 (màu xanh biển).

Để dễ hiểu tại sao lại chính xác hơn, hãy quan sát hình minh họa 6 và 7. Hai hình này ta thấy sự khác biệt giữa việc chỉ dùng gradient và kết hợp thêm hessian:

- **Hình 6** Đường xấp xỉ (màu xanh biển) là tiếp tuyến tại điểm dự đoán hiện tại. Nó chỉ dựa vào **độ dốc** g_i , tức hướng giảm nhanh nhất, mà bỏ qua độ cong của hàm. Do đó, giá trị tối ưu ước lượng có thể lệch khỏi cực tiểu thực sự nếu hàm mất mát cong nhiều.
- **Hình 7** Đường xấp xỉ (màu xanh biển) là dạng parabol khi dùng cả g_i và h_i . Lúc này ta không chỉ biết hướng mà còn biết hàm “cong” như thế nào quanh điểm hiện tại. Việc bổ sung hessian giúp mô hình điều chỉnh bước nhảy chính xác hơn, dẫn đến việc chọn cách phân nhánh và trọng số lá w_j sát với cực tiểu toàn cục hơn.

Nói cách khác, Gradient Boosting giống như đi xuống dốc chỉ dựa vào hướng nhìn thấy trước mắt, còn XGBoost vừa nhìn dốc vừa ước lượng độ cong để dự đoán điểm dừng hợp lý hơn. Đây chính là lý do XGBoost thường hội tụ nhanh hơn và cho kết quả ổn định hơn so với GB truyền thống.

II.5. Trọng số nút lá và Gain

Tương tự GB, sau khi cây f_t được xây dựng, ta cần gán giá trị cho các lá. Với lá j , tập dữ liệu trong lá được ký hiệu là I_j . Hàm mất mát (công thức số 3) sau khi được loại bỏ hằng số $l(y_i, \hat{y}_i^{(t-1)})$ để đơn giản hóa và thay $\Omega(f) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2$, sau khi thu gọn, ta được có công thức như sau:

$$\tilde{\mathcal{L}}^{(t)} = \sum_{j=1}^T \left[\left(\sum_{i \in I_j} g_i \right) w_j + \frac{1}{2} \left(\sum_{i \in I_j} h_i + \lambda \right) w_j^2 \right] + \gamma T. \quad (4)$$

Tối ưu hoá theo w_j cho ta trọng số tối ưu tại lá j với công thức tính:

$$w_j^* = - \frac{\sum_{i \in I_j} g_i}{\sum_{i \in I_j} h_i + \lambda}. \quad (5)$$

Thay công thức w_j^* vào hàm mất mát vừa đơn giản hóa (công thức số 4), ta được tổng lợi ích của toàn cây:

$$\tilde{\mathcal{L}}^{(t)}(q) = - \frac{1}{2} \sum_{j=1}^T \frac{\left(\sum_{i \in I_j} g_i \right)^2}{\sum_{i \in I_j} h_i + \lambda} + \gamma T. \quad (6)$$

Khi xét việc chia một nút lá thành hai nút con (trái và phải), chúng ta cần một thước đo để đánh giá xem việc tách này có thực sự làm giảm hàm mục tiêu hay không. Thước đo này được gọi là **Gain**, được tính bằng mức độ cải thiện của hàm mục tiêu trừ đi phần phạt cho việc thêm một nút lá mới:

$$\text{Gain} = \frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda} \right] - \gamma, \quad (7)$$

trong đó G_L, H_L và G_R, H_R lần lượt là tổng gradient và hessian của các điểm thuộc nút con trái và phải. Một phép tách sẽ được thực hiện nếu **Gain** > 0 , nghĩa là lợi ích thu được lớn hơn chi phí phải trả (γ). Ngược lại, nút sẽ không được tách và trở thành nút lá cuối cùng.

II.6. Các bước tính toán tổng quát

II.6.1. Phát biểu bài toán

Cho tập dữ liệu huấn luyện $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$, với $x_i \in \mathbb{R}^n$ và $y_i \in \mathcal{Y}$ ($\mathcal{Y} = \mathbb{R}$ cho hồi quy, $\mathcal{Y} = \{c_1, \dots, c_K\}$ cho phân loại). Ký hiệu $l(y, \hat{y})$ là hàm mất mát. Mỗi cây f có điều chuẩn:

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2,$$

trong đó T là số lá và w_j là trọng số của lá j .

XGBoost huấn luyện theo dạng cộng dồn:

$$F_M(x) = F_0(x) + \sum_{t=1}^M \eta f_t(x), \quad 0 < \eta \leq 1.$$

Tại vòng t , tối ưu

$$\mathcal{L}^{(t)} = \sum_{i=1}^N l(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)) + \Omega(f_t),$$

và dùng xấp xỉ Taylor bậc hai quanh $\hat{y}_i^{(t-1)}$:

$$\mathcal{L}^{(t)} \approx \sum_{i=1}^N \left[l(y_i, \hat{y}_i^{(t-1)}) + g_i f_t(x_i) + \frac{1}{2} h_i f_t(x_i)^2 \right] + \Omega(f_t), \quad (8)$$

với

$$g_i = \left. \frac{\partial l(y_i, \hat{y})}{\partial \hat{y}} \right|_{\hat{y}=\hat{y}_i^{(t-1)}}, \quad h_i = \left. \frac{\partial^2 l(y_i, \hat{y})}{\partial \hat{y}^2} \right|_{\hat{y}=\hat{y}_i^{(t-1)}}.$$

Vì $l(y_i, \hat{y}_i^{(t-1)})$ là giá trị mất mát từ vòng trước và không phụ thuộc vào cây mới $f_t(x_i)$, nó được xem như một hằng số trong bước tối ưu này và có thể được lược bỏ khỏi hàm mục tiêu để đơn giản hóa.

II.6.2. Step-by-step tổng quát cho XGBoost

1. Khởi tạo F_0 :

$$F_0(x) = \arg \min_{\theta} \sum_{i=1}^N l(y_i, \theta).$$

Ví dụ:

- Regression với hàm mất mát MSE: $F_0 = \frac{1}{N} \sum_i y_i$.
- Classification nhị phân với hàm mất mát logistic: $F_0 = \log \frac{\bar{y}}{1-\bar{y}}$ với $\bar{y} = \frac{1}{N} \sum_i y_i$.

2. Lặp với $t = 1, \dots, M$:

(a) **Tính thống kê bậc 1–2 (gradient, hessian):**

$$g_i = \left. \frac{\partial l(y_i, \hat{y})}{\partial \hat{y}} \right|_{\hat{y}=F_{t-1}(x_i)}, \quad h_i = \left. \frac{\partial^2 l(y_i, \hat{y})}{\partial \hat{y}^2} \right|_{\hat{y}=F_{t-1}(x_i)}.$$

Ví dụ:

- Regression (MSE): $g_i = \hat{y}_i - y_i$, $h_i = 1$.
- Binary classification (logistic): $g_i = p_i - y_i$, $h_i = p_i(1 - p_i)$ với $p_i = \sigma(\hat{y}_i)$.

(b) **Xây cây f_t :**

- Tại một nút chứa tập mẫu I : đặt $G = \sum_{i \in I} g_i$, $H = \sum_{i \in I} h_i$.
- Với mỗi đặc trưng k , sắp xếp toàn bộ các mẫu trong node theo giá trị của đặc trưng k . Ban đầu đặt $G_L = 0, H_L = 0$ và $G_R = G, H_R = H$. Sau đó, thử từng ngưỡng là trung bình các cặp giá trị từ trái sang phải, mỗi ngưỡng tính toán gradient và hessian của các mẫu bên trái và bên phải: G_L, H_L, G_R, H_R .
- Tính gain:

$$\text{Gain} = \frac{1}{2} \left(\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda} \right) - \gamma.$$

- Giữ lại ngưỡng có gain lớn nhất. Nếu $\text{gain} \leq 0$ hoặc đạt ngưỡng dừng, biến nút thành nút lá. Ngược lại, tách nút và lặp lại.

(c) **Tính trọng số lá:** Với mỗi lá j có tập I_j :

$$G_j = \sum_{i \in I_j} g_i, \quad H_j = \sum_{i \in I_j} h_i, \quad w_j^* = -\frac{G_j}{H_j + \lambda}.$$

(d) **Cập nhật mô hình:**

$$F_t(x) = F_{t-1}(x) + \eta f_t(x),$$

trong đó $f_t(x) = w_{q_t(x)}$ là trọng số lá mà x rơi vào.

3. **Kết quả cuối cùng:** sau M vòng, ta thu được mô hình $F_M(x)$.

II.7. Bài toán cụ thể

II.7.1. Regression

Xét hàm mất mát MSE: $l(y, \hat{y}) = \frac{1}{2}(y - \hat{y})^2 \Rightarrow \frac{\partial l}{\partial \hat{y}} = \hat{y} - y$, $\frac{\partial^2 l}{\partial \hat{y}^2} = 1$.

Thiết lập:

- Dữ liệu gồm 1 đặc trưng và có 6 mẫu như sau:

$$(x_i, y_i) \in \{(0, 0.0), (1, 1.0), (2, 1.5), (3, 2.5), (4, 3.0), (5, 4.0)\}.$$

- Siêu tham số: $\eta = 0.5$, $\lambda = 1.0$, $\gamma = 0.0$, $\text{max_depth} = 1$.

Bước 1: Khởi tạo F_0 . Với MSE, F_0 là giá trị trung bình của y :

$$F_0 = \frac{1}{6} \sum_{i=1}^6 y_i = 2.0.$$

Bước 2: Tính gradient & hessian tại vòng $t = 1$. Vì $\hat{y}_i^{(0)} = F_0 = 2.0$ cho mọi i , ta có

$$g_i = \hat{y}_i^{(0)} - y_i = [2, 1, 0.5, -0.5, -1, -2], \quad h_i = 1 \text{ (cho mọi điểm)}.$$

Bước 3: Tìm ngưỡng tách nút. Các ngưỡng tách thử trên x :

$$0.5, 1.5, 2.5, 3.5, 4.5.$$

Với mỗi ngưỡng, tính tổng trái/phải: $G_L = \sum g_i$, $H_L = \sum h_i$, G_R , H_R , và *gain*:

$$\text{Gain} = \frac{1}{2} \left(\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda} \right) - \gamma.$$

Với $\lambda = 1, \gamma = 0$:

Ngưỡng	G_L	H_L	G_R	H_R	Gain
0.5	2.0	1	-2.0	5	1.3333
1.5	3.0	2	-3.0	4	2.4000
2.5	3.5	3	-3.5	3	3.0625
3.5	3.0	4	-3.0	2	2.4000
4.5	2.0	5	-2.0	1	1.3333

Sau khi tính thử trên toàn bộ ngưỡng, ta được ghi nhận được ngưỡng 2.5 có gain lớn nhất.

Bước 4: Trọng số lá tối ưu. Phân nhánh tại ngưỡng 2.5, ta được:

$$G_L = 3.5, H_L = 3, G_R = -3.5, H_R = 3$$

$$w_L^* = -\frac{G_L}{H_L + \lambda} = -\frac{3.5}{3 + 1} = -0.875, \quad w_R^* = -\frac{G_R}{H_R + \lambda} = -\frac{-3.5}{3 + 1} = +0.875.$$

$$\text{Như vậy } f_1(x) = \begin{cases} -0.875, & x \leq 2.5, \\ +0.875, & x > 2.5. \end{cases}$$

Bước 5: Cập nhật dự đoán. Vì $\eta = 0.5$:

$$\hat{y}_i^{(1)} = \hat{y}_i^{(0)} + \eta f_1(x_i) = \begin{cases} 2 + 0.5 \times (-0.875) = 1.5625, & x_i \leq 2.5, \\ 2 + 0.5 \times (+0.875) = 2.4375, & x_i > 2.5. \end{cases}$$

Suy ra

$$\hat{y}^{(1)} = [1.5625, 1.5625, 1.5625, 2.4375, 2.4375, 2.4375].$$

II.7.2. Binary Classification

Xét hàm mất mát logistic:

$$l(y, \hat{y}) = -[y \log p + (1 - y) \log(1 - p)], \quad p = \sigma(\hat{y}) = \frac{1}{1 + e^{-\hat{y}}}.$$

Công thức cho gradient và hessian cho hàm mất mát logistic:

$$g_i = p_i - y_i, \quad h_i = p_i(1 - p_i).$$

Thiết lập:

- **Dữ liệu:** $(x_i, y_i) \in \{(0, 0), (1, 0), (2, 0), (3, 1), (4, 1), (5, 1)\}$.
- **Siêu tham số:** $\eta = 0.5, \lambda = 1.0, \gamma = 0.0, \text{max_depth} = 1$.

Bước 1: Khởi tạo F_0 . Tỷ lệ nhãn dương $= \frac{3}{6} = 0.5 \Rightarrow F_0 = \log \frac{0.5}{0.5} = 0, p = 0.5$.

Bước 2: Tính gradient & hessian tại vòng $t = 1$.

$$g = [0.5, 0.5, 0.5, -0.5, -0.5, -0.5], \quad h_i = 0.25.$$

Bước 3: Tìm ngưỡng tách nút. Các ngưỡng: 0.5, 1.5, 2.5, 3.5, 4.5.

Với $\lambda = 1.0, \gamma = 0.0$, ta có bảng tính gain cho từng ngưỡng như sau:

Ngưỡng	G_L	H_L	G_R	H_R	Gain
0.5	0.5	0.25	-0.5	1.25	0.1556
1.5	1.0	0.50	-1.0	1.00	0.5833
2.5	1.5	0.75	-1.5	0.75	1.2857
3.5	1.0	1.00	-1.0	0.50	0.5833
4.5	0.5	1.25	-0.5	0.25	0.1556

Gain lớn nhất tại ngưỡng 2.5 với:

$$G_L = 1.5, H_L = 0.75, \quad G_R = -1.5, H_R = 0.75.$$

Bước 4: Trọng số lá.

$$w_L = -\frac{1.5}{0.75 + 1} = -0.8571, \quad w_R = -\frac{-1.5}{0.75 + 1} = +0.8571.$$

Bước 5: Cập nhật dự đoán. Ban đầu $\hat{y}^{(0)} = 0 \Rightarrow p = 0.5$. Sau cập nhật ($\eta = 0.5$), các giá trị logit mới là:

$$\hat{y}_i^{(1)} = \hat{y}_i^{(0)} + \eta f_1(x_i) = \begin{cases} 0 + 0.5 \times (-0.8571) = -0.4286, & x \leq 2.5, \\ 0 + 0.5 \times (+0.8571) = +0.4286, & x > 2.5. \end{cases}$$

Từ đó, ta có thể tính xác suất mới tương ứng:

$$\begin{cases} p = \sigma(-0.4286) = 1/(1 + e^{0.4286}) \approx 0.394, & x \leq 2.5, \\ p = \sigma(+0.4286) = 1/(1 + e^{-0.4286}) \approx 0.606, & x > 2.5. \end{cases}$$

Suy ra, vector dự đoán (logit) sau vòng 1 là:

$$\hat{y}^{(1)} = [-0.4286, -0.4286, -0.4286, +0.4286, +0.4286, +0.4286].$$

Tổng kết

Qua hai ví dụ Regression và Classification, có thể thấy quy trình huấn luyện của XGBoost theo từng vòng đều gồm các bước:

1. Khởi tạo F_0 từ nhãn thật (giá trị trung bình đối với Regression, log-odds với xác suất của nhãn dương đối với Classification).
2. Tính gradient g_i và hessian h_i từ hàm mất mát tương ứng.
3. Xét các ngưỡng tách, tính gain để chọn ngưỡng tối ưu.
4. Tính trọng số lá w^* cho từng nhánh.
5. Cập nhật dự đoán $\hat{y}^{(t)} = \hat{y}^{(t-1)} + \eta f_t(x)$.

Bảng 1: So sánh Regression và Classification trong XGBoost.

	Regression (MSE)	Classification (Logistic)
Hàm mất mát	$\frac{1}{2}(y - \hat{y})^2$	$-[y \log p + (1 - y) \log(1 - p)]$
Gradient g_i	$\hat{y} - y$	$p - y$
Hessian h_i	1	$p(1 - p)$
Khởi tạo F_0	Trung bình y	$\log \frac{p}{1-p}$ với $p = \frac{\#1}{n}$
Kết quả sau vòng 1	$\hat{y}^{(1)}$ gần dần y	Xác suất p tách hai nhóm 0/1 rõ ràng hơn

III. Thực hành

III.1. Mô tả dữ liệu Heart Disease

Bộ dữ liệu **UCI Heart Disease** [4] là một trong những bộ dữ liệu y tế kinh điển, được sử dụng rộng rãi trong các nghiên cứu về học máy và khai phá dữ liệu. Với khoảng 303 bệnh nhân, với nhiều thuộc tính lâm sàng và kết quả chẩn đoán.

Bảng mô tả đặc trưng bộ dữ liệu Cleveland Heart Disease

Đặc trưng	Mô tả và giá trị mã hóa
age	Tuổi của bệnh nhân (năm).
sex	Giới tính. (1 = nam, 0 = nữ)
cp	Loại đau ngực. (1 = đau thắt ngực điển hình, 2 = không điển hình, 3 = đau không do tim, 4 = không triệu chứng)
trestbps	Huyết áp tâm thu lúc nghỉ (mmHg).
chol	Nồng độ cholesterol huyết thanh (mg/dL).
fbs	Đường huyết lúc đói > 120 mg/dL. (1 = đúng, 0 = sai)
restecg	Điện tâm đồ lúc nghỉ. (0 = bình thường, 1 = bất thường ST-T, 2 = phì đại thất trái)
thalach	Nhịp tim tối đa đạt được (lần/phút).
exang	Đau ngực khi gắng sức. (1 = có, 0 = không)
oldpeak	Mức độ trầm ST do gắng sức so với nghỉ.
slope	Độ dốc đoạn ST. (1 = dốc lên, 2 = bằng phẳng, 3 = dốc xuống)
ca	Số mạch máu chính được nhuộm màu (0–3).
thal	Thalassemia. (3 = bình thường, 6 = tổn thương cố định, 7 = tổn thương có thể đảo ngược)
num	Nhân mục tiêu. (0 = không bệnh, 1 = có bệnh)

Trong các ví dụ thực hành, để đơn giản hoá tính toán và minh hoạ rõ ràng các bước của

XGBoost, chúng ta sẽ chỉ chọn một vài đặc trưng liên tục như **age**, **chol**; và lấy một tập con nhỏ gồm vài bệnh nhân thay vì toàn bộ dữ liệu.

III.1.1. Regression với dữ liệu Heart Disease

Thiết lập: Chọn thuộc tính **age** làm biến đầu vào X và **chol** (cholesterol) làm biến mục tiêu y . Lấy một tập con nhỏ gồm 6 bệnh nhân:

$$X = [29, 48, 39, 67, 45, 59], \quad y = [204, 234, 203, 269, 250, 260].$$

Siêu tham số: $\eta = 0.5$, $\lambda = 1.0$, $\gamma = 0.0$.

Bước 1: Khởi tạo. Dự đoán ban đầu là trung bình của y :

$$F_0 = \frac{1}{6} \sum_{i=1}^6 y_i = 236.67.$$

Bước 2: Tính gradient & hessian tại vòng $t = 1$. Với loss MSE: $l(y, \hat{y}) = \frac{1}{2}(y - \hat{y})^2$, ta có

$$g_i = \hat{y}_i - y_i, \quad h_i = 1.$$

Kết quả:

$$g = [32.67, 2.67, 33.67, -32.33, -13.33, -23.33].$$

Bước 3: Tìm ngưỡng tách nút. Các ngưỡng ứng viên: 34.0, 42.0, 46.5, 53.5, 63.0. Bảng gain:

Ngưỡng	G_L	H_L	G_R	H_R	Gain
34.0	32.67	1	-32.67	5	355.70
42.0	66.33	2	-66.33	4	1173.36
46.5	53.00	3	-53.00	3	702.25
53.5	55.67	4	-55.67	2	826.34
63.0	32.33	5	-32.33	1	348.48

Ngưỡng có gain cao nhất $x = 42.0$.

Bước 4: Trọng số lá.

$$w_L = -\frac{66.33}{2+1} = -22.11, \quad w_R = -\frac{-66.33}{4+1} = 13.27.$$

Bước 5: Cập nhật dự đoán. Ban đầu $\hat{y}^{(0)} = 236.67$. Sau khi thêm cây (với $\eta = 0.5$):

$$\hat{y}_i^{(1)} = \hat{y}_i^{(0)} + \eta f_1(x_i) = \begin{cases} 236.67 + 0.5 \times (-22.11) = 225.61 & , x \leq 42, \\ 236.67 + 0.5 \times 13.27 = 243.30 & , x > 42. \end{cases}$$

Suy ra

$$\hat{y}^{(1)} = [225.61, 243.30, 225.61, 243.30, 243.30, 243.30].$$

III.1.2. Binary Classification với dữ liệu Heart Disease

Thiết lập: Chọn đặc trưng `age` làm đầu vào X , nhãn `target` (0/1) làm đầu ra y . Tập con dữ liệu:

$$X = [29, 48, 39, 67, 45, 59], \quad y = [0, 1, 0, 1, 1, 0].$$

Siêu tham số: $\eta = 0.5$, $\lambda = 1.0$, $\gamma = 0.0$.

Bước 1: Khởi tạo. Tỷ lệ nhãn dương = $\frac{3}{6} = 0.5 \Rightarrow F_0 = \log \frac{0.5}{0.5} = 0$, suy ra $p = 0.5$.

Bước 2: Tính gradient & hessian tại vòng $t = 1$. Vì hàm mất mát là logistic, ta có $g_i = p_i - y_i$ và $h_i = p_i(1 - p_i) = 0.5(1 - 0.5) = 0.25$. Theo thứ tự X xuất hiện ban đầu:

$$g = [0.5, -0.5, 0.5, -0.5, -0.5, 0.5], \quad h_i = 0.25.$$

Bước 3: Tìm ngưỡng tách nút. Các ngưỡng ứng viên: 34.0, 42.0, 46.5, 53.5, 63.0.

Ví dụ với ngưỡng 42.0, $G_L = 0.5 + 0.5 = 1.0$, $G_R = -0.5 - 0.5 + 0.5 - 0.5 = -1.0$:

Ngưỡng	G_L	H_L	G_R	H_R	Gain
34.0	0.50	0.25	-0.50	1.25	0.1556
42.0	1.00	0.50	-1.00	1.00	0.5833
46.5	0.50	0.75	-0.50	0.75	0.1429
53.5	0.00	1.00	0.00	0.50	0.0000
63.0	0.50	1.25	-0.50	0.25	0.1556

Ngưỡng tốt nhất là $x = 42.0$.

Bước 4: Trọng số lá. Với $G_L = 1.0$, $H_L = 0.5$ và $G_R = -1.0$, $H_R = 1.0$:

$$w_L = -\frac{G_L}{H_L + \lambda} = -\frac{1.0}{0.5 + 1.0} = -0.667, \quad w_R = -\frac{G_R}{H_R + \lambda} = -\frac{-1.0}{1.0 + 1.0} = 0.50.$$

Bước 5: Cập nhật dự đoán. Ban đầu $\hat{y}^{(0)} = 0$. Sau cập nhật với $\eta = 0.5$, các giá trị logit mới là:

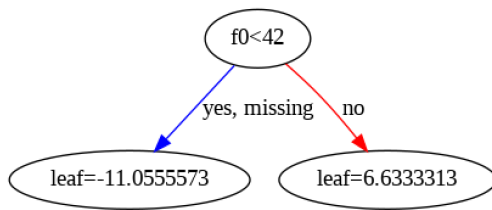
$$\hat{y}_i^{(1)} = \hat{y}_i^{(0)} + \eta f_1(x_i) = \begin{cases} 0 + 0.5 \times (-0.667) = -0.3335, & x \leq 42, \\ 0 + 0.5 \times (0.50) = 0.25, & x > 42. \end{cases}$$

Từ đó, ta có thể tính xác suất mới tương ứng:

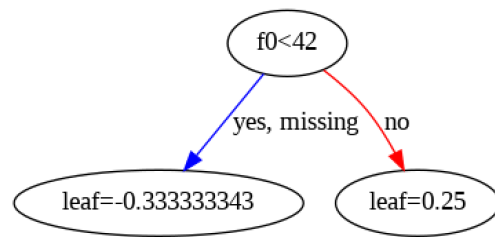
$$\begin{cases} p = \sigma(-0.3335) \approx 0.417, & x \leq 42, \\ p = \sigma(0.25) \approx 0.562, & x > 42. \end{cases}$$

Suy ra, vector dự đoán (logit) sau vòng 1, tương ứng với X ban đầu, là:

$$\hat{y}^{(1)} = [-0.3335, 0.25, -0.3335, 0.25, 0.25, -0.3335].$$



Hình 8: Kết quả sau khi chạy XGBoost bằng thư viện. Giá trị của các nút lá trong hình là giá trị đã được nhân với $\eta = 0.5$ do cách lưu giá trị của thuật toán. Kết quả trùng với kết quả w_L, w_R được tính thủ công ở III.1.1.



Hình 9: Kết quả sau khi chạy XGBoost bằng thư viện. Giá trị của các nút lá trùng với kết quả w_L, w_R được tính thủ công ở III.1.2.

IV. Câu hỏi trắc nghiệm

1. Trong XGBoost, thành phần nào của hàm mục tiêu phạt *số lượng lá*?

- (a) $l(y, \hat{y})$.
- (b) $\frac{\lambda}{2} \sum_j w_j^2$.
- (c) γT .
- (d) $\eta f_t(x)$.

2. Khi khai triển Taylor bậc hai trong XGBoost, g_i, h_i lần lượt là:

- (a) Sai phân bậc nhất và bậc hai của $\hat{y}$.
- (b) gradient và hessian của $\Omega(f)$.
- (c) gradient và hessian của $l(y_i, \hat{y})$ tại $\hat{y} = F_{t-1}(x_i)$.
- (d) Hai siêu tham số điều chuẩn.

3. Trọng số lá tối ưu (với điều chuẩn L2 λ) là:

- (a) $w_j^* = \frac{\sum_{i \in I_j} g_i}{\sum_{i \in I_j} h_i}$.
- (b) $w_j^* = -\frac{\sum_{i \in I_j} g_i}{\sum_{i \in I_j} h_i + \lambda}$.
- (c) $w_j^* = -\frac{\sum g_i}{\lambda}$.
- (d) $w_j^* = \frac{\sum h_i}{\sum g_i + \lambda}$.

4. Công thức Gain cho một phép tách trái-phải trong XGBoost là:

- (a) $\frac{G_L}{H_L} + \frac{G_R}{H_R} - \frac{G_L + G_R}{H_L + H_R}$.
- (b) $\frac{1}{2} \left(\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda} \right) - \gamma$.
- (c) $\frac{G_L^2 + G_R^2}{H_L + H_R}$.
- (d) $\frac{(G_L + G_R)^2}{(H_L + H_R)^2} - \gamma$.

5. Tác dụng chính của γ trong XGBoost là:

- (a) Giảm tốc độ học (learning rate).
- (b) Ràng buộc số lượng đặc trưng.
- (c) Ngưỡng cắt tỉa: chỉ chấp nhận split khi Gain $> \gamma$.
- (d) Điều chỉnh L1 trên w_j .

age	trestbps	target
52	140	0
58	150	0
45	130	0
60	145	1
62	160	1

Bảng 2: Bảng dữ liệu Heart Disease.

Thiết lập bài toán

- **Cho bài toán Regression:** Dùng hàm mất mát **MSE**, sử dụng đặc trưng **age** để dự đoán mục tiêu **trestbps**.
- **Cho bài toán Classification nhị phân:** Dùng hàm mất mát **logistic**, sử dụng đặc trưng **age** để dự đoán mục tiêu **target**.
- **Siêu tham số:** $\eta = 0.1$, $\lambda = 1.5$, $\gamma = 0$, chỉ 1 vòng boosting.

6. (Classification) F_0 được khởi tạo bằng?

- (a) $F_0 = 0.41$.
- (b) $F_0 = -0.41$.
- (c) $F_0 = 0.59$.
- (d) $F_0 = -0.59$.

7. (Classification) Ngưỡng có gain cao nhất là:

- (a) 48.5.
- (b) 55.0.
- (c) 59.0.
- (d) 61.0.

8. (Classification) Sau khi cập nhật, nếu cho một giá trị $x = 58$ thì $F(x)$ gần bằng:

- (a) 0.387.
- (b) -0.460 .
- (c) -0.345 .
- (d) 0.415.

9. (Regression) Trọng số nút lá w_L (nhánh bên trái) gần bằng:

- (a) 4.44.
- (b) -5.71 .
- (c) -4.44 .
- (d) 5.71.

10. (Regression) Sau khi cập nhật, nếu cho một giá trị $x = 52$ thì $F(x)$ gần bằng:

- (a) 144.43.
- (b) 145.44.
- (c) 154.34.
- (d) 135.44.

V. Tài liệu tham khảo

- [1] L. Grinsztajn et al., *Why do tree-based models still outperform deep learning on tabular data?*, 2022. arXiv: 2207.08815 [cs.LG].
- [2] Y. Freund et al., “A short introduction to boosting”, 1999.
- [3] T. Chen et al., “Xgboost: A scalable tree boosting system”, in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ser. KDD '16, ACM, Aug. 2016, 785–794.
- [4] S. W. P. M. Janosi Andras et al., *Heart Disease*, UCI Machine Learning Repository, 1989.

Phụ lục

1. **Datasets:** Các file dataset được đề cập trong bài có thể được tải tại [đây](#).
2. **Hint:** Các file code gợi ý có thể được tải tại [đây](#).
3. **Demo:** Chương trình demo cho nội dung trong bài có thể được truy cập tại [đây](#).
4. **Solution:** Các file code cài đặt hoàn chỉnh và phần trả lời nội dung trắc nghiệm có thể được tải tại [đây](#).
5. **Q&A:** Bạn có thể đặt thêm câu hỏi về nội dung bài đọc trong group Facebook hỏi đáp tại [đây](#). Tất cả câu hỏi sẽ được trả lời trong vòng tối đa 4 tiếng.

AIO_QAs-Verified

🔒 Private group · 1.4K members



Hình 10: Hình ảnh group facebook AIO Q&A.